MORTIS

Anonymous Authors[1]

ABSTRACT

Existing agent serving systems exploit the predictability of agentic workloads to guide KV-cache scheduling but only at a binary prefix–suffix granularity. We observe that agent prompts have finer schedulable structure, a property we call *intra-prompt heterogeneity*: they can be decomposed into multiple heterogeneous components that differ in sharing pattern and lifetime. This property creates opportunities to enhance KV-cache management by exposing more prefix reuse and scheduling KV-cache more precisely. Meanwhile, frameworks that let LLM invocations be declared as language constructs emit prompts programmatically, making intra-prompt heterogeneity analyzable from program code. To this end, we present MORTIS, a compiler-serving co-design that recovers and exploits intra-prompt heterogeneity for efficient KV-cache management. Our evaluation shows that MORTIS reduces average JCT by 1.45× and raises throughput by 1.49× on geometric mean over vanilla SGLang, and by up to 1.99× and 1.97×. On a mixed workload, it raises throughput by 1.56× on geometric mean and by up to 1.86×.

1 INTRODUCTION

Agent applications accomplish tasks through a series of dependent LLM requests that often share long contexts. A recent characterization reports that 85–97% of prompt tokens are shared across requests (Srivatsa et al., 2025). Serving one LLM request involves a prefill phase that computes the key-value (KV) cache for the prompt, followed by autoregressive decoding. Prefill is particularly expensive for long prompts because attention cost grows quadratically with prompt length, but the extensive context sharing in agent applications creates substantial opportunities to avoid redundant prefill computation by reusing the KV cache. However, KV cache reuse cannot be taken for granted: limited GPU memory means that not every reusable KV cache can be retained. Agent serving systems must therefore manage the KV cache carefully, as losing reusable entries is costly: once the KV cache thrashes, per-request prefill time can grow by 6.8× and generation throughput can drop by up to 66% (Yuan et al., 2026).

Fortunately, agent workloads are far more predictable than generic chat traffic. Agent serving systems exploits this predicablity to schedule KV-cache. However, they operate on a binary view of prompt: a reusable prefix followed by an opaque main prompt (surfix). Scheduling decisions therefore do not make full use of agent behavior patterns and extensive context sharing, as they lose information internal to prompts.

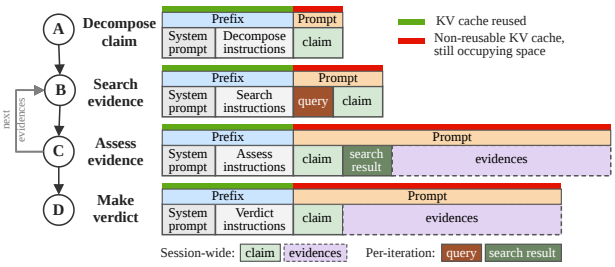


Figure 1. A fact-checking agent that verifies a *claim*'s correctness. Its prompts can be decomposed into heterogeneous components with different sharing patterns and lifetimes. Without this information, the serving system misses opportunities for KV-cache reuse and cannot precisely identify and evict stale KV-cache entries.

However, we observe that agent prompts have a finer structure that can guide scheduling. A prompt is a composition of heterogeneous components whose **sharing pattern** and **lifetime** differ. We call this property **intra-prompt heterogeneity**. The fact-checking agent in Figure 1 exhibits it: **(i)** *sharing pattern*: *claim* is shared by all four calls, whereas *evidences* is shared only by calls C and D; **(ii)** *lifetime*: *claim* and *evidences* live for the whole session, with *evidences* growing every iteration, whereas *query* and *search_result* are replaced in each loop iteration.

This property opens two optimization opportunities. **First**, more prefix caching can be exposed by laying out components in prompts. Because prefix reuse stops at the first different token, placing stable, widely shared components

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

at the front keeps a divergent token from cutting off the reusable prefix, such as placing shared *claim* at the beginning. Among the agents we evaluated (§4), 51–92% of the shared tokens cannot reuse the KV-cache because of such invalidation. **Second**, the KV-cache can be scheduled more precisely. KV-cache can be scheduled at the granularity of components rather than treating them as monolithic units. In our workload we observed that about 20% of served prompt tokens are only used once within one session. For example, KV-caches for *query* and *search_result* are dead after their last use, which should be cleaned.

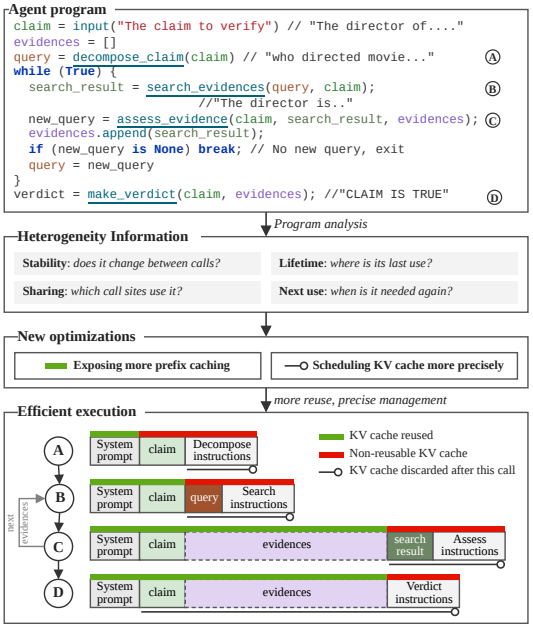


Figure 2. Intra-prompt heterogeneity is analyzable from code, this property can be exploited for more efficient serving.

Exploiting these opportunities requires discovering intra-prompt heterogeneity. **We argue that intra-prompt heterogeneity information resides in agent program itself, which can be discovered with compiler techniques.** Agent frameworks like byLLM, NOOA, DSpy (Furgale et al., 2026; Khattab et al., 2024; Dantanarayana et al., 2025) expose LLM invocations through programming abstractions, letting developers declare LLM invocations as functions with typed parameters. Then the framework runtime generates the prompts from this declarations and the argument values. Consequently, components within prompts are mapped to each variable, making sharing patterns and lifetimes encoded in program itself. Intra-prompt heterogeneity is therefore analyzable. As Figure 2 shows, the program itself reveals intra-prompt heterogeneity and the derived information guides more efficient KV-cache scheduling.

To this end, we present MORTIS, a compiler-serving co-design that recovers and exploits intra-prompt heterogeneity. MORTIS uses *adaptive prompt re-layout* and *proactive KV-cache scheduling* to expose more prefix-cache reuse and to schedule scarce cache memory more efficiently. However, exploiting intra-prompt heterogeneity raises several challenges. **First, the comprehensive recovery needs both code information and actual runtime behaviors.** Static analysis can provide basic structure, but knowledge on behavior comes from runtime. MORTIS bridge the gap by combining information from both sides with its compiler-serving co-design. **Second, prompt layout is local to each call-site, but maximizing prefix reuse requires coordination across the whole agent.** MORTIS uses a five-level tagging system and analyze based on workflow to determine the final layout to maximize prompt reuse. **Third, KV-cache scheduling consumes the resourses it tries to save.** Retention and prefetching hold memory that other requests could use, yet evicting too eagerly forces recomputation. MORTIS protects the prefixes most likely to be reused within a fixed memory budget, and prefetches only into memory reclaimed from dead KV-cache.

In summary, we make the following contributions: **(i)** We identify *intra-prompt heterogeneity* in agent workloads and the further optimization opportunities it offers. **(ii)** We propose a methodology that uses compiler techniques to recover intra-prompt heterogeneity and guide efficient KV-cache management with that property. **(iii)** We design and implement MORTIS, a compiler-serving co-design that guides KV-cache scheduling with the recovered intra-prompt heterogeneity. Our evaluation shows that MORTIS reduces average JCT by 1.45× and raises throughput by 1.49× on geometric mean over vanilla SGLang, and by up to 1.99× and 1.97×. On a mixed workload, it raises throughput by 1.56× on geometric mean and by up to 1.86×.

2 BACKGROUD AND MOTIVATIONS

2.1 LLM inference and LLM agent serving

2.1.1 *KV-cache and prefix caching*

KV-cache Modern decode-only LLMs split their execution into two main phases (Kamath et al., 2025): a compute-intensive **Prefill Phase** that processes the input prompt in parallel to produce the first token and a Key-Value (KV) cache, and a memory-bandwidth-bound **Decode Phase** that autoregressively generates subsequent tokens one by one, using the KV cache. The KV-Cache stores attention key and value states to prevent redundant computations.

Prefix caching For requests that share common prompt prefixes, systems like SGLang (Zheng et al., 2024) and vLLM (Kwon et al., 2023) maintain radix trees mapping token sequences to their computed KV caches. When a new request arrives, the serving system searches the tree for the

longest matching prefix and reuses its KV cache, only computing the remaining tokens. However, prefix caching stops at the first different token, because causal attention makes each token's KV states depend on all preceding tokens. Hierarchical caching systems such as CachedAttention (Gao et al., 2024), Mooncake (Qin et al., 2025), and SGLang HiCache (SGLang Contributors, 2026) extend the KV-cache beyond device memory with host memory and optionally disk storage. KV-cache that is not in the device tier should be promoted before using.

2.1.2 *KV-cache management for LLM agent*

Agent serving systems use recurring patterns in agentic workloads to manage KV-cache. KVFlow (Pan et al., 2025) derives each agent's steps-to-execution from the workflow graph to guide eviction and prefetching; Helium (Wadlom et al., 2026) analyzes the workflow to order LLM calls and prewarm static prefixes; PBKV (Zheng et al., 2026) and CacheScout (Zhang et al., 2026) predict upcoming agent invocations with trained models to decide which caches to retain. Continuum (Li et al., 2025) pinned KV-cache across tool gap. They conduct management operations like prefetching, eviction, and protection on the identified prefix cache. Some systems, such as Pythia (Yu et al., 2026), futhermore prefill the predicted next invocations to proactively prepare KV-cache that does not exist yet.

2.2 Frameworks with LLM calls as typed functions

A class of agent frameworks exposes model-backed computation through typed programming interfaces. ByLLM's meaning-typed programming (MTP) abstraction (Dantanarayana et al., 2025), DSPy's Signatures and Modules (Khattab et al., 2024), and NOOA's typed Python methods (Furgale et al., 2026). The developer declares a function or signature describing the desired behavior, inputs, and return type, and invokes it like an ordinary call within the program's control flow. At each invocation, the framework runtime lowers the declaration and its argument values into messages that send to model.

Although these agent frameworks differ in surface syntax, they lower LLM invocations in similar ways. Developers declare LLM invocations as functions or function-like declarations, and the framework serializes that declaration together with the runtime arguments into a single prompt. Their generated prompts carry same functional components. Their generated prompts carry the same functional components. As shown in Figure 3, these frameworks emit the same five kinds of components: framework scaffolding, task instruction, parameter descriptions, argument values, and output schema. The only difference is their relative positions.

Scheduing Opportunity: The order of the five components carries no intent. It is decided solely by the framework

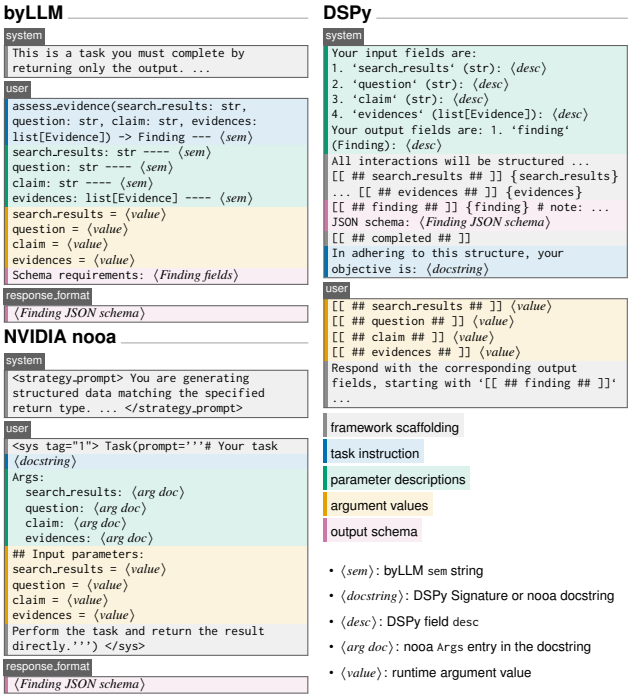


Figure 3. Prompts lowered by three frameworks for the same call share five kinds of components; only their order differs.

runtime, without any program-specific knowledge. Moreover, the order of argument values is just an artifact of the developer's personal preference: they follow the order of the argument list in the code; Therefore these layout can be managed for optimized performance. Frameworks have begun to exploit this freedom for cache efficiency. For example, DSpy's static adapter (DSPy Contributors, 2026) let user tweak lowering process to put stable prompt at the front. NOOA (Furgale et al., 2026) lets developers label arguments as static prefix, append-only history or dynamic suffix so that the framework can order them for better cache reuse.

2.3 Intra-prompt heterogeneity in program

As we discussed in §1, prompts can be decomposed into components with different sharing patterns and lifetimes, and frameworks with LLM calls as typed functions offer structural guarantees to make this information analyzable. Because components are mapped to arguments, and arguments are variables in the program, the semantic boundaries and relationships are in program.

Scheduling Opportunity: With the analyzable sharing patterns and lifetimes, we can figure out the operations for exact KV-cache entries, such as deprecating KV-cache that will not be reused and protecting KV-cache that will be. Existing serving systems only schedule the matched prefix, losing potential for better performance.

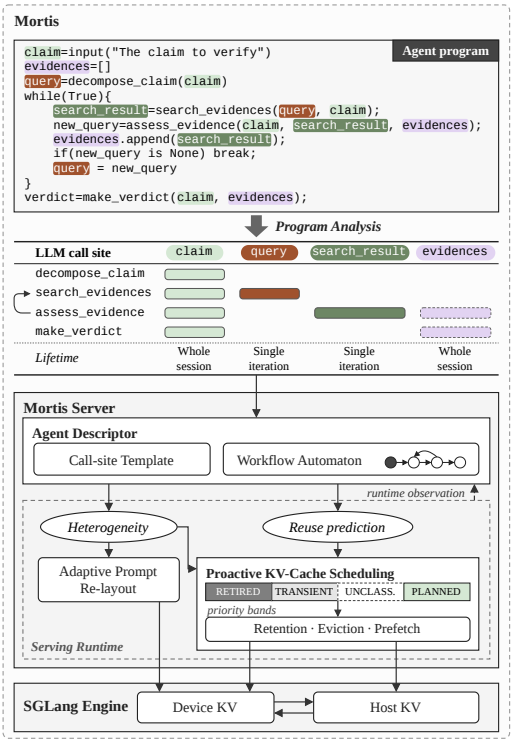


Figure 4. Architecture overview of MORTIS. By analyzing data that already resides in program itself, MORTIS steers a more efficient KV-cache scheduling policy.

2.4 MORTIS: compiler-serving co-design

3 DESIGN

In this section, we present the two-tier compiler-serving co-design of MORTIS. As shown in Figure 4, *agent descriptor* is generated by the combination of analyzing agent program and observing runtime behaviors. Server side then exploits the derived descriptor to schedule the KV-cache efficiently, using *adaptive prompt re-layout* (§3.2) and *proactive KV-cache scheduling* (§3.3).

3.1 Agent Descriptor

Agent descritor contains the metadata for scheduling. Static analysis provides it a skeleton and the server refines the weights with runtime observations. An agent descriptor contains two parts: (i) **Call-site Template:** It provides a heuristic of the components' lifetime, sharing pattern. (ii) **Workflow Automaton:** It contains the program-level structure of an agent, including workflow dependencies, execution time estimation and transition probabilities.

MORTIS traverses the agents type-checked AST to identify LLM call sites and their relationships. It then classifies each parameter's provenance and lifetime to construct the skeleton of the agent descriptor.

3.1.1 *Call-site Template*

Prompts can be decomposed into heterogeneous components, we classify them into the following two categories: (i) **Invariant Specifications:** the components that are fixed once the program is compiled, such as the system prompt, response schema, and the tool declarations. They are rendered from the compiler's own intermediate representation and stay constant. (ii) **Dynamic Bindings:** because LLM calls are declared as ordinary functions, the arguments of a call site are supplied at runtime. These runtime values are the dynamic bindings.

We define the call-site template as $\mathcal{T}_c = \langle \mathcal{I}_c,\ \mathcal{B}_c,\ \mathcal{H}_c \rangle$, where $\mathcal{I}_c$ is the invariant specifications, $\mathcal{B}_c$ is the dynamic bindings, and $\mathcal{H}_c$ is the heterogeneity annotation over those bindings. All three are recovered by static analysis.

Heterogeneity annotation. For each dynamic binding, $\mathcal{H}_c$ records both how its value is shared and how long that value stays valid: $\mathcal{H}_c : \mathcal{B}_c \longrightarrow \mathcal{E} \times \mathcal{S}$. The sharing pattern component $\mathcal{E}$ classifies a binding by its value provenance. We distinguish $\mathcal{E}$ into five classes, listed from the most to the least reusable: **(i) CONST:** the value is fixed by the program text and is identical at every call. **(ii) COPY:** the value is the same as the value another call site carries, so the bytes are already resident once that call site has run. **(iii) RESP:** the value comes from the reply produced by another call site. **(iv) EXTEND:** the binding is an append-only context, and the previous value remains a prefix of the new one. **(v) VOLATILE:** the value is freshly produced at each call, or its origin lies outside what the analysis can see. We write ρ for the rank of a class in the order above, which expresses how much of a binding's bytes an earlier call can already supply.

The evolution component $\mathcal{E}$ governs which bytes a later prompt can reuse, while the lifetime component $\mathcal{S}$ governs when cached bytes can be given up.

3.1.2 *Workflow Automaton*

An agent typically consists of a sequence of dependent LLM calls whose workflow structure is encoded in the program. Concretely, we abstract an agent as a time-annotated probabilistic finite-state machine (TPFM). We formulate the TPFM as:

$$\mathcal{A} = (Q, P, q_0, F, \mathcal{T}),$$

where: (i) Q is the set of states. Each state $q \in Q$ is labeled by an LLM callsite; (ii) P denotes the transition probabilities between states. (iii) $q_0 \in Q$ is the entrance callsite. (iv) F specifies the termination probability of the agent at each state. (v) $\mathcal{T} = \left\{ D_q^{\text{exec}}, D_{q,q'}^{\text{gap}} \right\}$ denotes the timing annotations associated with states and transitions, where D_q^{exec} is the execution-time distribution of the LLM

callsite represented by state q, and $D^{\text{gap}}_{q,q'}$ is the inter-call gap distribution between the completion of callsite q and the invocation of its successor q'.

Static analysis fixes the shape of this automaton, while its weights come from runtime logs. The server observes $\mathcal{T}$ and P online and accumulates them across sessions, updating the statistical model of the agent consistently.

3.2 Adaptive Prompt Re-layout

The message a framework emits for a call site follows the declaration order of the function, which may place a changing value ahead of values that later calls could reuse. MORTIS's adaptive re-layout permutes the components of each call message so that the bytes most likely to be resident already come first.

3.2.1 Movable Components

First, the dynamic bindings $\mathcal{B}_c$ can be permuted freely. The compiler lowers each argument into a self-contained component labeled by its parameter name, and the framework places these components in declaration order only because that is the order of the argument list; re-layout may choose any order of the bindings.

Second, part of the invariant specification $\mathcal{I}_c$ is movable. $\mathcal{I}_c$ consists of the system prompt, the response schema, the tool schema, and the instruction specification (such as task descriptions and parameter descriptionss). The instruction specification is movable, as system prompt stays constant and schemas are consumed by structured structured generation engine separately.

3.2.2 Layout Decision

Each call site sorts the bindings it carries by a program-wide key. Generally, *shared* bindings, which are carried by multiple call sites, are placed at the front, followed by the bindings carried by a single call site. Algorithm 1 summarizes the procedure.

Sorting private bindings. The bindings carried by a single call site are sorted in ascending lexicographic order of:

$$K(b) = \big(\textit{scoped},\ \rho,\ \textit{depth},\ -\textit{length}\big),$$

where: **(i)** *scoped*: the lifetime of b; 0 if it is session-wide ($\mathcal{S} = \emptyset$) and 1 otherwise, so bindings that survive scope boundaries precede those reset by them; **(ii)** ρ: the stability rank defined in §3.1.1, from 1 for CONST to 5 for VOLATILE, so the more reusable class leads; **(iii)** *depth*: The number of transitions to the first call site that carries it from entrance, so values that enter a session earlier lead; **(iv)** *length*: the token length of the value of b, observed by the server; negated so longer bindings precede.

Algorithm 1 Layout decision algorithm

Require: Descriptor $(\{\mathcal{T}_c\}_{c\in C}, \mathcal{A})$, lengths ℓ, switch *pin*
Ensure: Layouts $\{L_c\}_{c\in C}$

$B_{\text{sh}} \leftarrow \{b : |\{c \in C : b \in \mathcal{B}_c\}| > 1\}$; $R \leftarrow \emptyset$
for all $e = (q, q') \in \textsc{Edges}(\mathcal{A})$ **do**
  $B_e \leftarrow B_{\text{sh}} \cap (\mathcal{B}_q \cup \mathcal{B}_{q'})$
  $t_e(b) \leftarrow 1$ if *unchanged*, 2 if *append-only*, else 3
  $R \leftarrow R \cup \{(b, b') \in B_e^2 : t_e(b) < t_e(b')\}$
$J(b) \leftarrow (-\ell(b), \textit{depth}(b))$; $J_{\min}(S) \leftarrow \min_{b\in S} J(b)$
$D \leftarrow \textsc{ClusterConflicts}(B_{\text{sh}}, R)$
$\langle S_1, \ldots, S_k\rangle \leftarrow \textsc{TopoSort}(D, J_{\min})$ ▷ Min-key first
$\pi_{\text{sh}} \leftarrow \textsc{Sort}(S_1, J) \| \cdots \| \textsc{Sort}(S_k, J)$
for all $c \in C$ **do**
  $\pi_c \leftarrow \langle b \in \pi_{\text{sh}} : b \in \mathcal{B}_c\rangle \| \textsc{Sort}(\mathcal{B}_c \setminus B_{\text{sh}}, K)$
  $s_c \leftarrow \textsc{InstructionSpec}(\mathcal{I}_c)$
  $L_c \leftarrow \langle s_c\rangle \| \pi_c$
  if $\neg$*pin* $\wedge$ $(\mathcal{B}_c \cap B_{\text{sh}} \neq \emptyset)$ **then** $L_c \leftarrow \pi_c \| \langle s_c\rangle$
return $\{L_c\}_{c\in C}$

Sorting shared bindings. For every transition from q to q', we identifies how each shared binding of the two call sites evolves. We categorize these patterns into three groups, ordered as follows: (**i**) Both call sites carry it and its value does not change between the two calls. (**ii**) Both call sites carry it and the program only appends to its value. (**iii**) Program rewrites it between the two calls, or only one of the two call sites carries it.

MORTIS collects these requirements over all transitions and orders the shared bindings to satisfy them; when several bindings can go next, the one with the smallest (***-length***, ***depth***) is placed first. Bindings with conflicting requirements are clustered and sorted by the same key.

Placing the specification. The *instruction specification* is lowered from code by the compiler and is unique to each call site. Emitting it first would prevent prefix sharing among different call sites. MORTIS can move the specification to the end to share larger common prefix when the chance exists. An switch can keep the specification at the front when developer requires.

3.3 Proactive KV-Cache Scheduling

MORTIS's KV-cache scheduling proactively arranges the placement of KV-cache entries. Using the agent descriptor, the server decides operations for every served KV-cache sequence.

3.3.1 Reusable Head and Volatile Tail

A served prompt naturally divides into two segments: a reusable head and a volatile tail. The reusable head consists

of two parts: a *static prefix* and a *session-shared extension*: (i) **Static Prefix** is the prefixes that don't change. including $\mathcal{I}_c$ and the CONST bindings that re-layout has placed behind $\mathcal{I}_c$. (ii) **Session-shared Extension** is the bindings behind the static prefix that a later call of the same session will emit again identically.

3.3.2 Next-Call Prediction

Our prediction focuses on the following three aspects: (i) The probability of the arrival. (ii) When the call will arrive. (iii) Which prefix of its prompt can be speculatively constructed.

Arrival probability. We estimate the probability of the next call by a loop-tagged table. MORTIS adopts Bourdoncle-style (Bourdoncle, 1993) weak topological ordering to discover all loops within the agent based on the emitted workflow graph. Each call-site q can be located by a chain of loops enclosing it; for each call-site's invocation, the server marks it with a tuple $\zeta = \{i_1, i_2, \ldots, i_n\}$, where i_k is the current iteration index of the k-th enclosing loop, outermost first. We define the table as $\mathcal{P}(q' \mid q, \zeta)$ and $\mathcal{F}(q, \zeta)$, where $\mathcal{P}$ stands for the probability that a call-site q in context ζ is followed by a call-site q'; $\mathcal{F}$ stands for the probability that the session ends at call-site q under context ζ. With an empty ζ (no loop context), $\mathcal{P}$ and $\mathcal{F}$ coincide with P and F.

The table is a frequency count that the server updates across sessions. Let q and ζ denote the call-site and loop context of the preceding call in the same session. When a call at q' arrives, the server increments the count of outcome q' in row (q, ζ); when the session closes at q, it increment termination count in row (q, ζ); Each observation is also counted in the row of every suffix of ζ, down to the empty tuple, so that for the same q the table holds one row per level of context. After a call at q lands, we compute the probability of each outcome o by Witten–Bell interpolation from the empty context ζ_0 to the full context $\zeta_m = \zeta$, where o ranges over $\mathrm{succ}(q)$, the static successors of q plus a virtual outcome end for session termination:

$$\hat{P}_k(o) = \lambda_k \frac{c_k(o)}{n_k} + (1 - \lambda_k)\hat{P}_{k-1}(o), \quad k = 0, \ldots, m,$$

$$\hat{P}_{-1}(o) = \frac{1}{|\mathrm{succ}(q)|}, \qquad \lambda_k = \begin{cases} \dfrac{n_k}{n_k + d_k} & n_k > 0, \\ 0 & n_k = 0, \end{cases}$$

where ζ_k is the suffix of ζ that keeps only the innermost k indices, and the three counts are read from the row (q, ζ_k) of the table: $c_k(o)$ is how many times outcome o followed a call at q in context ζ_k, $n_k = \sum_o c_k(o)$ is the number of observations in that row, and d_k is the number of distinct outcomes with $c_k(o) > 0$.

Then $\mathcal{P}(q' \mid q, \zeta) = \hat{P}_m(q')$ and $\mathcal{F}(q, \zeta) = \hat{P}_m(\mathrm{end})$. A level with no observation is skipped. The weight λ_k grows with the support of a level, so a well-observed iteration dominates the prediction while a sparse one defers to the pooled behavior of q.

Arrival time. The time of arrival is conservatively approximated based on recorded statistics. The workflow automaton records $D^{\mathrm{gap}}_{q,q'}$ per transition: each time a call at q' arrives, the server appends the interval between the reply of the preceding call at q and this arrival. The interval is the agent's own time spent in its code between two calls.

We estimate the gap before the next call as:

$$\hat{g} = t_{50} - \tfrac{1}{2}(t_{90} - t_{50})$$

where t_{50} and t_{90} are the 50th and 90th percentiles of the gap samples on the transition (q, q'); the predicted call is then expected $\hat{g}$ after the reply of the current call. The second term ensures that a transition with a more dispersed gap distribution receives a tighter estimate.

Prefix reconstruction. MORTIS construct the prefix of call-site q' for matching KV-cache entries to schedule. As $\mathcal{H}_c$ tells the relationships of arguments, MORTIS builds prefix in layout Lq' using observed argument values in current session. The construction stops at the first argument with unknown value.

3.3.3 KV-cache Eviction Using Lifecycle Bands

Bands. We categorize KV-cache entries into four bands, from first evicted to last: (1) **RETIRED:** the volatile tails of prompts belonging to completed sessions. (2) **TRANSIENT:** the volatile tail of a served prompt of a live session. (3) **UNCLASSIFIED:** entries that analysis cannot categorize. They keep the engine's default treatment. (4) **PLANNED:** prefixes that a predicted call will reuse.

Eviction empties the lowest band first and falls back to recency only within a band, so KV-cache that will likely be reused leaves memory later. On the host tier it keeps LRU eviction except that retired segments go first, so a head the device gave up under pressure remains reloadable while it may still be needed. Additionally, KV-cache entries marked as *TRANSIENT* are not admitted into the host tier, as they wouldn't be reused.

Band transitions. Bands are assigned at three moments of a session's lifecycle. (1) **When a served prompt enters the cache**, MORTIS splits it at its reusable head (§3.3.1). The volatile tail and the reply generated after it become *TRANSIENT*; the head stays *UNCLASSIFIED* until the next prediction refresh finds a successor call that starts with it,

which promotes it to *PLANNED*. (2) **When a later call arrives**, the transition it takes may rewrite bindings that earlier heads carried. Each affected head is cut back to the prefix that survives the rewrite, and the bytes past it are demoted to *TRANSIENT*. (3) **When the session ends**, the volatile tails of every prompt it served drop to *RETIRED*. Their heads stay *UNCLASSIFIED* and age out by recency, since other sessions of the same program may still share them.

These bands are mirrored directly into the radix tree's eviction order. The band is the primary key of the eviction heap and recency breaks ties.

3.3.4 KV-cache Retention and Prefetch

Based on the prediction, MORTIS's conducts KV-cache protection, which ensures the required prefix's kv-cache exists. Each prediction is summarized into a tuple of reconstructed prefix, arrival probability and estimated deadline:

$$\textit{Prediction} = (\pi_{q'},\ \mathcal{P}(q' \mid q, \zeta),\ t_{\text{reply}} + \hat{g})$$

where $\pi_{q'}$ is the prefix reconstructed for q' and t_{reply} is the time the current call replied. Predictions are refreshed at every call, and are dropped when the predicted calls arrive or expire. Protection acts on this tuple in two ways: *Retention* keeps $\pi_{q'}$ from leaving the device, and *prefetch* restores the part of $\pi_{q'}$ that has already been demoted to the host.

Retention. MORTIS ranks the predicted reusable KV-cache entries by the benefit of keeping $\pi_{q'}$ resident at every refresh:

$$\mathcal{D}(q') = \frac{\mathcal{P}(q' \mid q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}}{\max(t_{\text{reply}} + \hat{g} - t_{\text{now}},\ \tau_{\min})}$$

where $|\pi_{q'}|_{\text{cached}}$ is the number of tokens of $\pi_{q'}$ cached (on either host or device) and $\tau_{\min}$ is a floor that keeps a call due now from dominating. MORTIS sorts all predictions by $\mathcal{D}$ in descending order, and admits them into *PLANNED* until the band covers a fixed fraction of the device tier. Within the band, their $\mathcal{D}$ add up on every radix cache node, so that a shared head outranks a private one.

Prefetch. When part of $\pi_{q'}$ has been demoted to the host, the planner prefetches it back to the device ahead of the estimated arrival time. A prefetch release deadline is defined as:

$$t_{\text{release}} = t_{\text{reply}} + \hat{g} - \frac{|\pi_{q'}|_{\text{host}}}{B_{\text{h2d}}} - \frac{|\pi_{q'}|_{\text{uncached}}}{B_{\text{prefill}}}$$

where B_{h2d} and B_{prefill} are the measured host-to-device copy rate and prefill rate. Among released predictions, the earliest deadline goes first, with ties broken by expected saving $\mathcal{P}(q' \mid q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}$. A prefetch is released when the engine is idle or its deadline t_{release} approaches. It takes only free device memory, after reclaiming *RETIRED* and *TRANSIENT* entries. When no space is free, prefetching pauses but keeps touching its entries on the host so they do not age out by recency.

4 EVALUATION

4.1 Experimental Setup

4.1.1 Testbed and Model

We implement MORTIS on top of SGLang v0.5.10.post1. Experiments are conducted on one A40 GPU using Qwen3-32B-AWQ. We allocate 15 GB of GPU memory to the device KV-cache pool and 25 GB of CPU memory to the host KV-cache pool.

4.1.2 Baselines

We compare MORTIS against the following serving systems. All baselines are reimplemented on MORTIS's SGLang fork with their original scheduling algorithms: (i) **Vanilla SGLang** (Zheng et al., 2024) uses SGLang's HiCache with LRU eviction, on the same SGLang version (v0.5.10.post1). (ii) **CacheScout** (Zhang et al., 2026) identifies agents from their request prefixes and learns inter-agent execution transitions online. It uses the learned transition patterns to prioritize cached KV entries and prefetch the prefixes of predicted successor agents. (iii) **Continuum** (Li et al., 2025) retains each agent program's KV-cache for an adaptively selected time-to-live (TTL). The TTL is derived from observed inter-invocation delays and expected cache reuse, allowing the system to preserve reusable context while the agent executes tools. (iv) **KVFlow** (Pan et al., 2025) uses an Agent Step Graph to build a workflow-aware KV-cache management framework for agentic workloads. (v) **Static-Adapter** mirrors the adapter layer of DSPy (DSPy Contributors, 2026), which applies a fixed, program-agnostic layout to every call, hoisting the invariant components to the front.

We also evaluate the following ablation variants of MORTIS: (i) **MORTIS-Relayout-only** enables adaptive prompt re-layout but replaces proactive KV-cache scheduling with LRU eviction; (ii) **MORTIS-No-Relayout** enables proactive KV-cache scheduling but disables adaptive prompt re-layout; (iii) **MORTIS-No-Prefetch** disables only KV-cache prefetching. In addition, we run the baseline systems systems on top of MORTIS's adaptive prompt re-layout.

4.1.3 Workloads

We evaluate MORTIS on four representative agent workloads, ported to Jac: a HoVer-style (Jiang et al., 2020) fact-checking agent, a HumanEval (Chen et al., 2021) coding agent, a BFCL (Patil et al., 2025) tool-use agent, and a

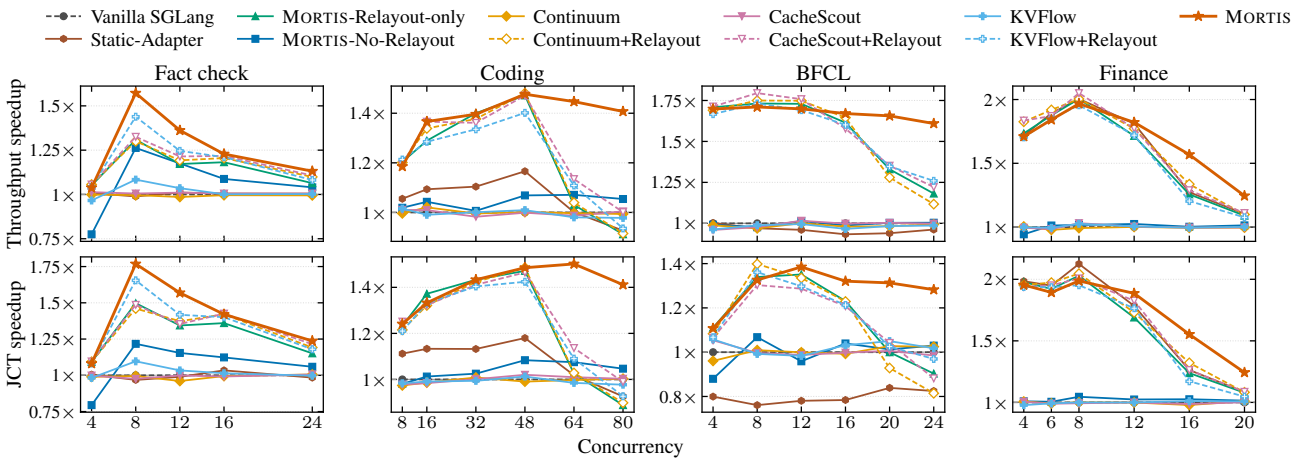


Figure 5. Throughput speedup (top) and average JCT speedup (bottom) over Vanilla SGLang on each agent workload under varying concurrency.

FinanceBench (Islam et al., 2023) financial-analysis agent. We further evaluate a mixed workload comprising all four agent types to assess MORTIS's end-to-end performance under concurrent, heterogeneous demand.

4.2 End-to-End Performance of MORTIS

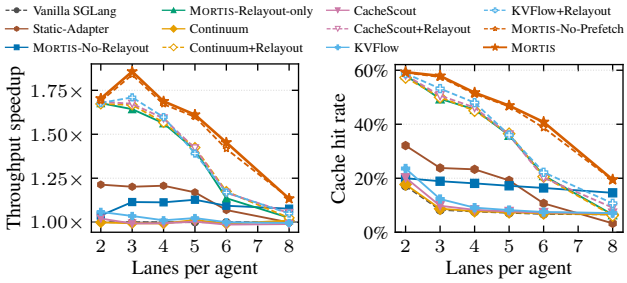
We test MORTIS under different numbers of concurrent workflows and on a mixed workload. Across these workloads, we compare average job completion time (JCT) and throughput (prompt tokens served per minute).

Performance on different workload types As shown in Figure 5, MORTIS reduces average JCT by 1.08–1.99× and increases throughput by 1.04–1.97× compared with Vanilla SGLang, with geometric-mean gains of 1.45× in JCT and 1.49× in throughput. Across these agents, frameworks such as KVFlow show no significant improvement, because they ignore the intra-prompt heterogeneity.

When concurrency is low, adaptive prompt re-layout accounts for nearly all of the improvement, because memory is sufficient. As load increases, however, the KV-cache becomes congested. Without knowing what to keep and what to give up, other frameworks fail to schedule it efficiently.

For the financial-analysis agent, MORTIS shows little improvement, because this agent is a simple pipeline that digests large blobs of documents. Existing frameworks handle this task well once adaptive prompt re-layout exposes prefix reuse. MORTIS only takes effect under the highest load, because it can proactively evict stale KV-cache entries.

Performance on mixed workload We evaluate MORTIS under varying pressure on the mixed workload and report metrics over each system's steady-state window. We vary the number of lanes per agent, keeping it equal across all four agents.



(a) Throughput speedup (b) Cache hit rate

Figure 6. Throughput speedup over Vanilla SGLang and cache hit rate under varying lanes per agent on the mixed workload.

As shown in Figure 6, MORTIS increases throughput to 1.86× that of Vanilla SGLang with 3 lanes per agent. Under low pressure, adaptive prompt re-layout alone accounts for nearly all of the gain, since device memory is still large enough to retain the reusable KV-cache. As pressure grows, the cache becomes contested and the gain from re-layout alone shrinks, while proactive KV-cache scheduling retains a clear advantage by protecting the reusable heads that LRU would evict. Under the heaviest pressure, the two configurations converge toward the baseline, as the engine is fully saturated and no free memory is left for cache management to exploit. Across the six lane counts, the geometric-mean throughput gain is 1.56×.

In the mixed workload, agents have heterogeneous demands for KV-cache memory, which reduces the chance that they contend for memory at the same time. By exploiting intra-prompt heterogeneity, MORTIS can promptly yield stale

Table 1. Task accuracy under three prompt layouts that appear in our evaluation.

Prompt layout	Fact-check	Coding agent	Doc analysis	BFCL agent
Original	16.3%	55.3%	34.9%	14.1%
Relayout	21.9%	54.8%	37.2%	18.7%
Adapter	16.3%	52.2%	37.8%	16.0%

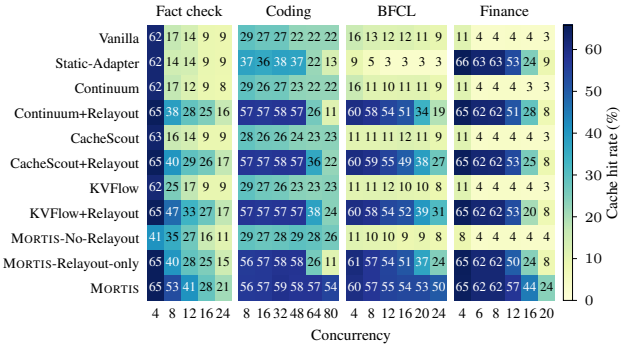


Figure 7. Cache hit rate (%) of prompt tokens under varying numbers of concurrent workflows.

KV-cache to other requests that need memory, which further amplifies the benefit of MORTIS.

Task Quality The performance gains of MORTIS do not come at the cost of task quality, since task accuracy is insensitive to the layout of prompt components(§**??**). As shown in Table 1, across three different layouts produced by the original byLLM, Static-Adapter, and MORTIS, task accuracy does not vary notably.

4.3 Cache Efficiency of MORTIS

Figure 7 shows the cache hit rate of prompt tokens at the same concurrency levels as Figure 5. On coding and BFCL, adaptive prompt re-layout alone brings the hit rate at low concurrency to that of MORTIS: it places the stable and shared bindings at the front, where they form a reusable head that later calls hit. As concurrency grows, the hit rate of MORTIS-Relayout-only drops quickly, because LRU eviction does not distinguish reusable heads from volatile tails. With proactive KV-cache scheduling, MORTIS keeps a higher hit rate than MORTIS-Relayout-only as concurrency grows, and on coding its hit rate stays nearly flat. On fact check, MORTIS's hit rate exceeds that of MORTIS-Relayout-only from the lowest concurrency. Continuum and CacheScout stay close to Vanilla SGLang at every concurrency. MORTIS-No-Relayout stays below MORTIS, since under the original prompt order the reusable head it can protect is short. The mixed workload shows the similar pattern (Figure 6).

Table 2. Geometric-mean speedup over Vanilla SGLang across all concurrency levels of the four agent workloads, with MORTIS's *Adaptive Prompt Re-layout* and *Proactive KV-Cache Scheduling* enabled or disabled.

(a) JCT speedup

	Scheduling	
Re-layout	Off	On
Off	1.00×	1.03×
On	1.30×	1.45×

(b) Throughput speedup

	Scheduling	
Re-layout	Off	On
Off	1.00×	1.02×
On	1.36×	1.49×

4.4 Ablation Analysis

Interactions beteen MORTIS's mechanisms We calculate the geometric-mean speedup of JCT and throughput over Vanilla SGLang on the four workloads. As shown in Table 2, the two mechanisms are superadditive: when both scheduling and re-layout are enabled, the speedup is higher than the product of the two individual speedups. The same effect appears on the mixed workload (Figure 6a), where adding proactive KV-cache scheduling on top of adaptive prompt re-layout substantially improves performance.

The gain of MORTIS's KV-cache scheduling builds on re-layout, because re-laying out the prompt order exposes a reusable prefix and a suffix that can be abandoned. MORTIS-No-Relayout enables only MORTIS's KV-cache scheduling, and its speedup stays low (Table 2), with the same trend in Figures 6 and Figure 5.

Prefetching contributes little on this testbed. As shown in Figure 6a, disabling it (MORTIS-No-Prefetch) lowers throughput by at most 2.6% across the six lane counts. The gain of proactive KV-cache scheduling mainly comes from retention, which keeps the reusable heads resident.

Baselines under the same prompt layout. As shown in Figure 6, baseline serving systems perform close to Vanilla SGLang when served with the original prompt layout, since only a small fraction of the prefix can be reused. After enabling MORTIS's adaptive prompt re-layout, these serving systems all show performance gains, even though they are still surpassed by MORTIS. This is because they manage the KV-cache only at the granularity of a prefix and its suffix.

4.5 Sensitivity study

We further run the mixed workload on four A40 GPUs with TP=4. Keeping the same KV-cache pool size, TP=4 increases the prefill bandwidth, which lowers the cost of a cache miss.

5 Related Work

Frameworks for serving LLM agents. KVFlow (Pan et al., 2025) schedules KV by distance to the next invocation on an application-supplied agent step graph. Continuum (Li et al., 2025), CacheScout (Zhang et al., 2026), and PBKV (Zheng et al., 2026) learn retention signals from execution history, and Pythia (Yu et al., 2026) reconstructs the workflow from traces to drive cache staging, scheduling, and autoscaling. All of them manage whole requests or agent steps. MORTIS instead recovers the structure encoded in the program, and manages the KV-cache at intra-prompt granularity.

Prompt layout and reordering. Existing systems treat prompt layout as a fixed convention or delegate it to a model. SGLang's frontend reorders prompt statements to lengthen the shared prefix, but relies on GPT-4 to choose the order, which its authors note is not yet reliable (Zheng et al., 2024). DSPy's adapters emit components in one fixed order across all programs (DSPy Contributors, 2026), and NOOA argues for a three-region layout that places static context first (Furgale et al., 2026). These orders are chosen by convention or by an unsound rewrite. MORTIS derives the layout by systematically analyzing the program itself.

KV reuse beyond exact prefixes. Prompt Cache fixes module positions in a schema (Gim et al., 2024), and CacheBlend selectively recomputes tokens when concatenating cached chunks (Yao et al., 2025). Both relax positional exactness to gain reuse, at the cost of soundness or extra recomputation.

6 Conclusion

The intra-prompt heterogeneity of programmatically generated prompts creates opportunities for optimization. MORTIS discovers these opportunities by static analysis assisted by runtime observations, and steers KV-cache management with this information through adaptive prompt re-layout and proactive KV-cache scheduling. Our evaluation demonstrates that MORTIS reduces average JCT by 1.45× on geometric mean and by up to 1.99× over Vanilla SGLang, raises throughput by 1.49× on geometric mean and by up to 1.97×. On the mixed workload, MORTIS improves throughput by 1.56× on geometric mean and by up to 1.86×.

References

Bourdoncle, F. Efficient chaotic iteration strategies with widenings. In *Formal Methods in Programming and Their Applications*, volume 735 of *Lecture Notes in Computer Science*, pp. 128–141. Springer, 1993. doi: 10.1007/BFb0039704.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dantanarayana, J. L., Kang, Y., Sivasothynathan, K., Clarke, C., Li, B., Kashmira, S., Flautner, K., Tang, L., and Mars, J. Mtp: A meaning-typed language abstraction for ai-integrated programming. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):1176–1204, 2025. doi: 10.1145/3763092.

DSPy Contributors. ChatAdapter: DSPy api reference. Online documentation, 2026. URL https://dspy.ai/api/adapters/ChatAdapter/. Accessed September 14, 2026.

Furgale, P., Klingler, S., Nolan, J., Staats, M., Di Lorenzo, G., Martinez Abad, E., Schüller, C., Dinu, R., Devoto, A., Berard, P., Kaplun, G., Sarafian, E., Roveri, R., Derczynski, L., and Silveira Cabral, R. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents, 2026. URL https://arxiv.org/abs/2607.20709.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 111–126, Santa Clara, CA, 2024. USENIX Association. URL https://www.usenix.org/conference/atc24/presentation/gao-bin-cost.

Gim, I., Chen, G., seob Lee, S., Sarda, N., Khandelwal, A., and Zhong, L. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 325–338, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.

Islam, P., Kannappan, A., Kiela, D., Qian, R., Scherrer, N., and Vidgen, B. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

Jiang, Y., Bordia, S., Zhong, Z., Dognin, C., Singh, M., and Bansal, M. HoVer: A dataset for many-hop fact extraction

and claim verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3441–3460, 2020.

Kamath, A. K., Prabhu, R., Mohan, J., Peter, S., Ramjee, R., and Panwar, A. POD-Attention: Unlocking full prefill-decode overlap for faster LLM inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '25, pp. 897–912. Association for Computing Machinery, 2025. doi: 10.1145/3676641.3715996.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3600006.3613165. URL https://arxiv.org/abs/2309.06180.

Li, H., He, R., Mang, Q., Zhang, Q., Mao, H., Chen, X., Zhou, H., Zhang, H., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live, 2025.

Pan, Z., Patel, A., Hu, Z., Shen, Y., Guan, Y., Li, W.-L., Qin, L., Wang, Y., and Ding, Y. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows, 2025. URL https://arxiv.org/abs/2507.07400.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025.

Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170, Santa Clara, CA, 2025. USENIX Association. URL https://www.usenix.org/conference/fast25/presentation/qin.

SGLang Contributors. HiCache system design and optimization. SGLang documentation, 2026. URL https://docs.sglang.io/docs/advanced_features/hicache_design. Accessed September 22, 2026.

Srivatsa, V., He, Z., Abhyankar, R., Li, D., and Zhang, Y. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=meKEKDhdnx.

Wadlom, N., Shen, J., and Lu, Y. Efficient LLM serving for agentic workflows: A data systems perspective, 2026. URL https://arxiv.org/abs/2603.16104.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3689031.3696098. URL https://www.microsoft.com/en-us/research/uploads/prod/2024/09/eurosys25-final999.pdf.

Yu, S., Shu, J., Ni, Y., Qian, K., Li, X., Wang, Y., Zhang, J., Xu, Z., Yang, S., Zhu, L., Zhai, E., Lu, Q., Xing, J., Lu, Y., Jin, X., Liu, X., and Xu, H. Pythia: Exploiting workflow predictability for efficient agent-native LLM serving, 2026. URL https://arxiv.org/abs/2604.25899.

Yuan, Y., Nayak, A., Kundu, S., and Talati, N. Agentic AI workload characteristics. In *2026 IEEE International Symposium on Workload Characterization (IISWC)*, 2026. URL https://arxiv.org/abs/2605.26297.

Zhang, R., Kim, C., Feng, S., Du, K., Liu, Y., Zhong, Y., Ching, C.-W., Jiang, J., and Hu, L. Learning agent execution for kv-cache management in agentic serving, 2026.

Zheng, H., Fu, F., Wu, J., Yuan, B., Zhang, Y., Wang, H., Zhu, Y., Yan, X., and Jiang, J. Efficient serving for dynamic agent workflows with prediction-based KV-cache management, 2026. URL https://arxiv.org/abs/2605.06472.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.